

CONF42 • DEVSECOPS 2026

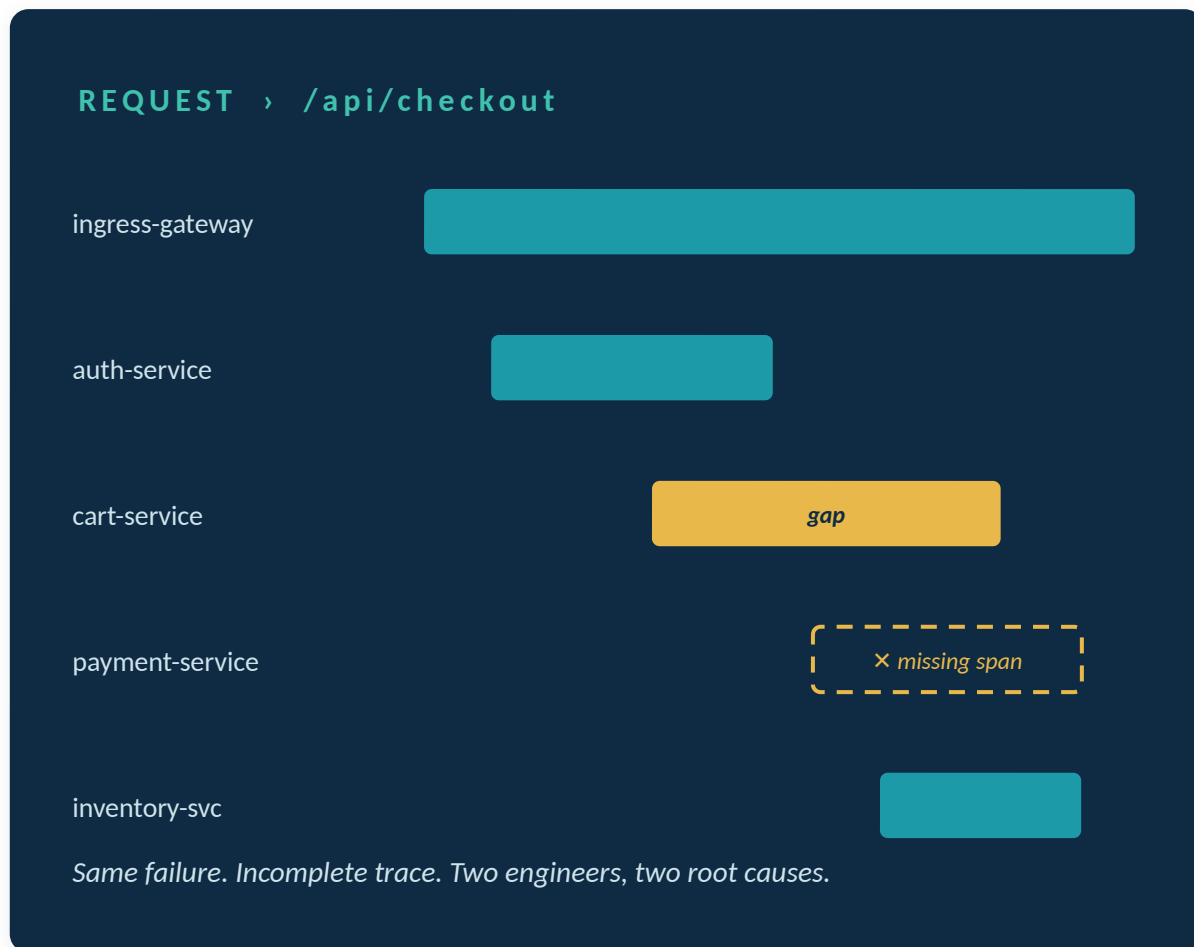
Deterministic Kubernetes Diagnostics with OpenTelemetry

Reliable, secure, end-to-end request tracing across enterprise microservices — for the DevSecOps engineer who owns reliability and security at once.

Presented by *Mridula Chilakamarri*



The Diagnostic Problem in Distributed Systems

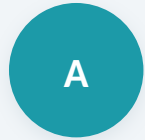


A single failed request can touch dozens of independently deployed microservices. When telemetry is fragmented — logs in one system, metrics in another, traces incomplete — debugging becomes non-deterministic: the conclusion depends on who is looking and which system they checked first.



Fragmented telemetry isn't an inconvenience — it's a compliance and incident-response risk. If you can't reconstruct what happened, you can't defend it to an auditor.

What This Talk Covers



Architecture

Design decisions that make request tracing deterministic — not best-effort.



Security

Allowlist-based metadata; alignment with SOC 2, HIPAA, PCI-DSS.



Resilience

Zero request impact when any telemetry component fails.



Verification

Seven capabilities you can independently test — in a pipeline, not by hope.

Everything here is testable — every claim is something you could check.

Why Standard Observability Falls Short



No shared identity

Collected separately, with no common thread linking related events across data types — so correlation is manual.

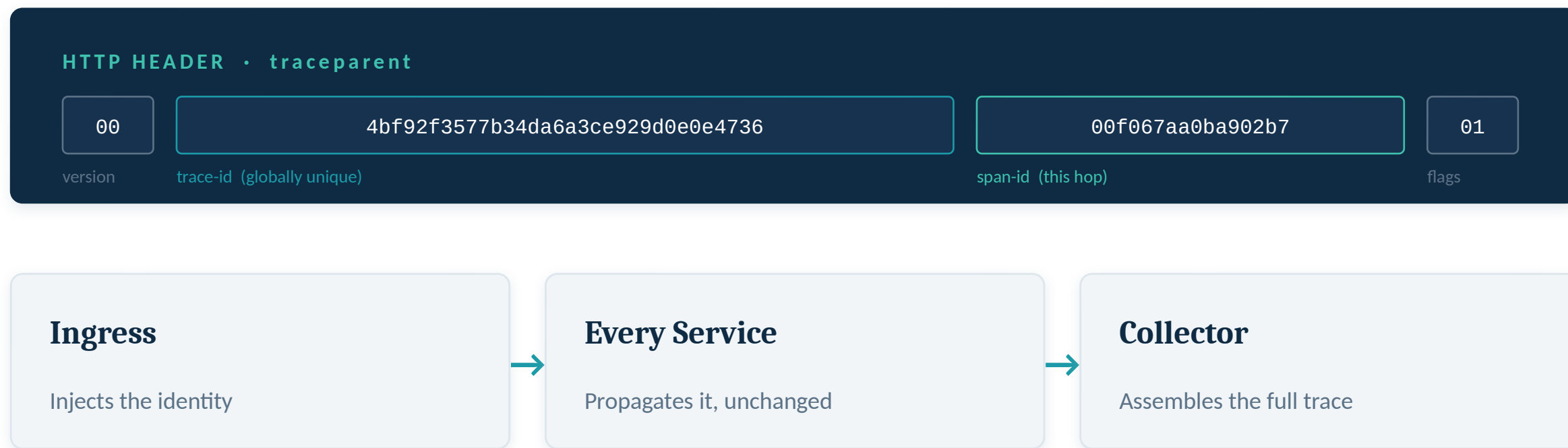
Conventional observability treats logs, metrics, and traces as separate domains. Without a shared request identity crossing service boundaries, correlation is manual, error-prone, and non-reproducible.

- No common trace identifier across service hops
- Manual log correlation — slow and unreliable
- Inconsistent sampling drops the exact failure you needed
- Telemetry gaps lead to the wrong root cause

The fix isn't a better dashboard — it's a shared identity.

The Foundation: W3C Trace Context

A vendor-neutral standard for propagating a trace ID and span ID across HTTP calls. One identity, injected once, surviving every hop — the base for deterministic, end-to-end diagnostics.



Interoperable by design — you're not locked to any one vendor's SDK, and the identity survives every hop.

Three Architecture Decisions

1

Ingress-level trace identity

Trace context is injected at the cluster ingress boundary — every inbound request carries a globally unique ID before it touches a single service.

2

Mandatory context propagation

Services are required — not encouraged — to forward trace headers. Optional propagation creates gaps, and a gap is exactly where your worst incidents hide.

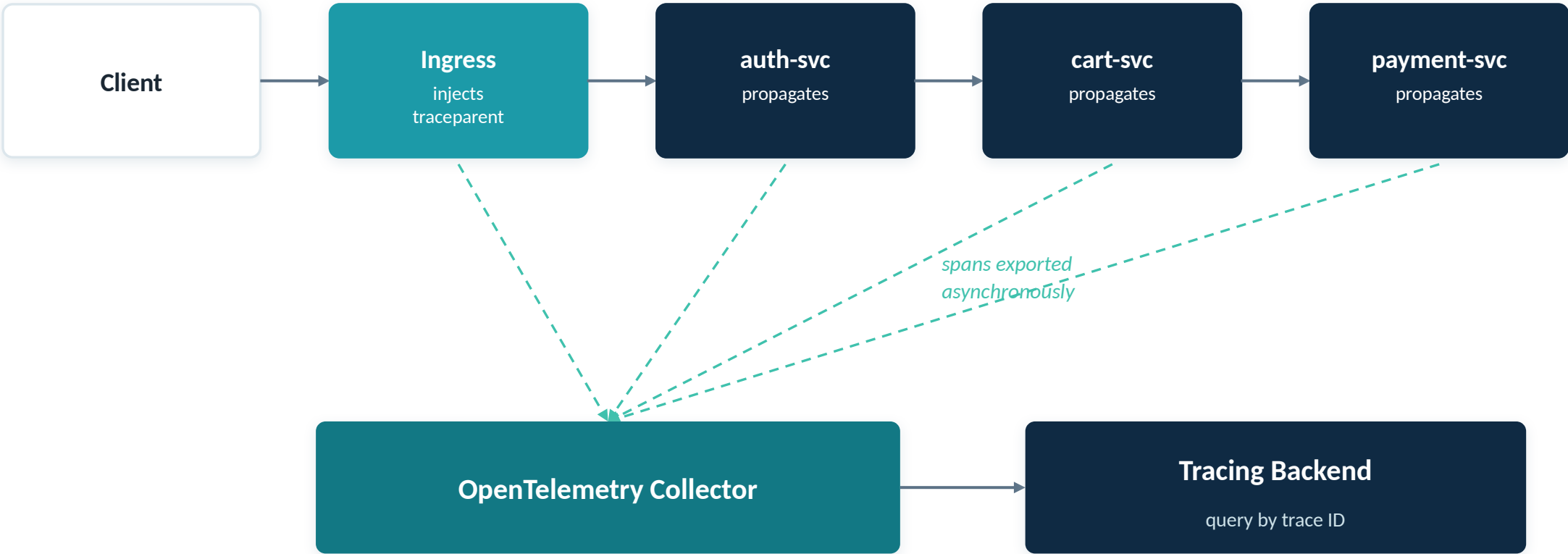
3

Configuration-driven controls

Sampling, export targets, and verbosity live in config, not code. Tune telemetry in production without a redeploy — minutes, not hours, during an incident.

End-to-End Request Path

REQUEST PATH (synchronous — trace ID flows unchanged)



Export is off the request path. If the collector or backend fails, spans are dropped — requests still succeed.

Allowlist-Based Metadata: Security by Design

Unrestricted metadata propagation is how PII, credentials, or internal topology leak into your telemetry pipeline. We invert the default: instead of blocking bad fields, we approve only vetted ones.

The default is deny

A field enters a trace only after it passes an explicit approval gate. Diagnostic richness without turning your trace store into a compliance liability.

NINE APPROVED OPERATIONAL FIELDS



trace ID



span ID



parent span ID



service name



operation name



timestamps



duration



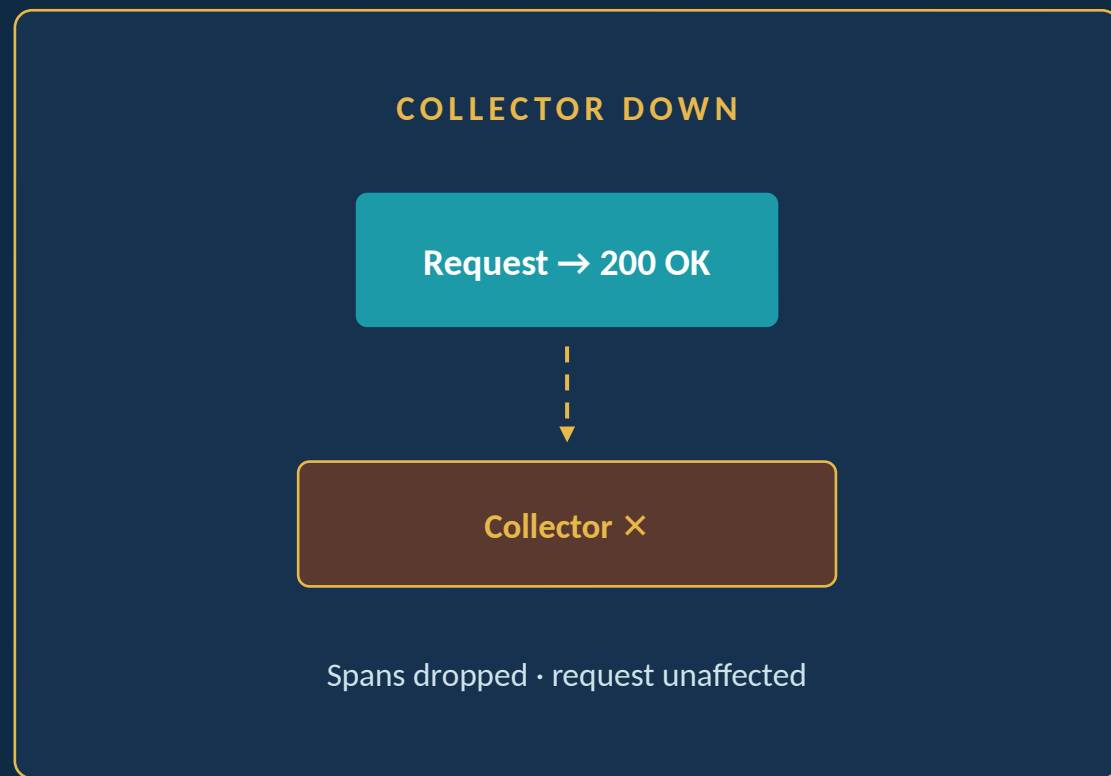
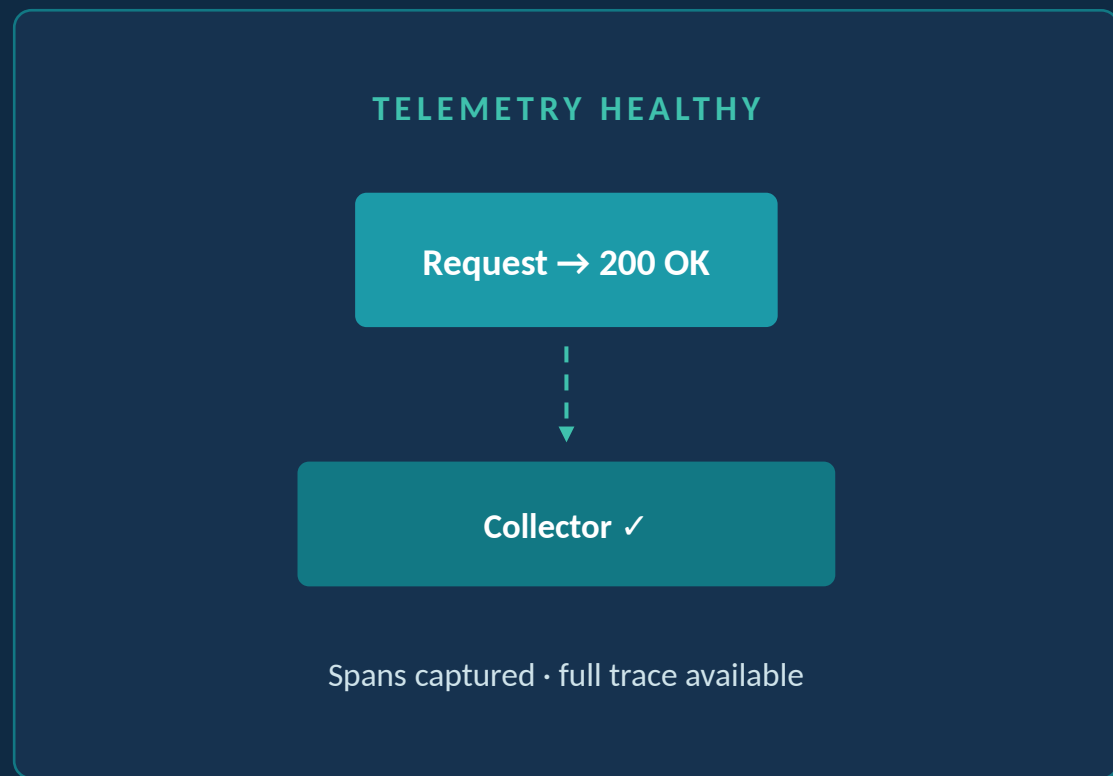
status



HTTP response
code

Everything else — payloads, headers, user data — never enters the pipeline.

Zero Request Impact on Telemetry Failure



Collectors, exporters, backends are non-critical-path dependencies. Observability data is lost; user traffic is not.

One Design, Three Compliance Frameworks

Because identity is consistent and metadata is controlled, the same architecture answers all three — you're not building three separate compliance stories.

SOC 2

Reproducible audit trail

Deterministic traces give auditors a consistent, replayable record of what happened during any event.

HIPAA

No PHI in telemetry

Allowlist excludes payloads and user data by design — protected health information never reaches the trace store.

PCI-DSS

No cardholder data in traces

Deny-by-default metadata means PANs and credentials can't ride along in span attributes.

Seven Capabilities the System Guarantees · 1-4

Reliable tracing — every request identified, every hop captured, every trace queryable.

1

Globally unique trace ID at ingress

Every request gets a unique trace ID; IDs already present on incoming requests are preserved and propagated unchanged.

2

Correct span hierarchy per service

Each service creates its own span with proper parent-child lineage. Retries preserve the original trace ID.

3

Full trace-level data capture

Every service records trace & span IDs, parent, service and operation names, timestamps, duration, status, and HTTP code.

4

Query any trace by ID

SREs look up a trace by ID and see the complete end-to-end path with service-to-service relationships.

Seven Capabilities the System Guarantees · 5–7

Operable & secure — config-driven, richly visualized, and safe by design.

5

Config-only export

Export destinations set via Kubernetes config — no application code changes. Multiple backends and tenant-specific routing supported.

6

Rich platform visualization

End-to-end trace views, service dependency graphs, span hierarchies, and per-service, per-span latency breakdowns.

7

Non-disruptive & secure

Tracing never blocks or fails requests when the backend is down. Trace data excludes payloads, credentials, and PII by design.

Each is independently testable — which means they're automatable in CI/CD. "Are we tracing-ready?" becomes a checklist you run, not a judgment call.

Production Lesson: One Incident, Two Wrong Answers

An illustrative scenario — the pattern this architecture is built to eliminate.

1 THE SYMPTOM

Checkout fails intermittently — roughly 1 in 20 requests returns a 500. No pattern in the dashboards. Reproducible only under load.

2 THE FRAGMENTED HUNT

Engineer A sees DB timeouts in cart-service logs and blames the database. Engineer B sees elevated latency on payment-service metrics and blames a downstream API. Both look at real signals — from the same failure — and reach opposite conclusions.

3 THE DETERMINISTIC ANSWER

Pull one failing trace by ID. The waterfall shows the truth: auth-service exhausts its connection pool under load, delaying every downstream hop. The DB timeout and payment latency were both symptoms — not the cause. One trace, one answer, minutes not hours.

What Production Taught Us

1

Constraints shaped the design

Mandatory propagation and ingress-level identity weren't the elegant choice — they were the choice that survived real services owned by real teams. Optional anything erodes to gaps.

2

Design failure in from day one

Non-disruptive telemetry failure can't be retrofitted. If export sits on the request path even once, you'll find out during your worst outage — not before.

3

Security is cheaper structural

Bolting metadata controls on after launch means auditing every existing field. Deny-by-default from the start is a one-time gate, not a perpetual cleanup.

Operational Benefits at Scale

7

Evaluation capabilities

Independently testable —
automatable in CI/CD

9

Approved metadata fields

Allowlisted for compliance-safe
diagnostic richness

3

Compliance frameworks

SOC 2, HIPAA & PCI-DSS from a
single design

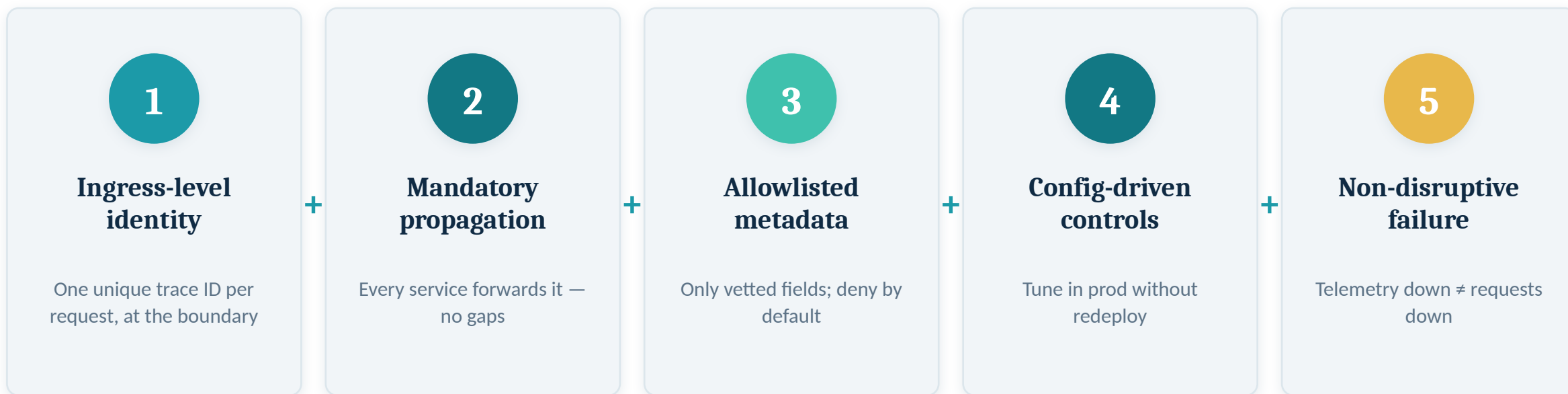
0

Request impact

On telemetry failure — resilience
is non-negotiable

Putting It Together

Any one of these is a nice-to-have. Together, they're what makes diagnostics deterministic.



= deterministic, secure, resilient request diagnostics

Key Takeaways



Determinism is architecture, not tooling

Mandatory propagation, ingress identity, and config discipline separate reliable tracing from aspirational instrumentation.



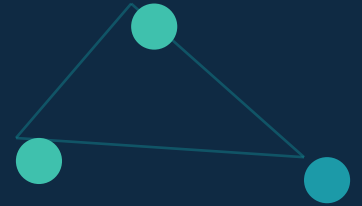
Security is structural, not additive

The allowlist is part of the design — not a filter you bolt on after the leak.



Resilience is a contract, not a goal

Zero request impact on telemetry failure must be enforced by architecture and verified — never assumed.



Thank you

Questions? Let's talk about the seven capabilities, the allowlist, or how any of this fits your stack.

Mridula Chilakamarri · Conf42 DevSecOps 2026

